

CMSC 478: Machine Learning

KMA Solaiman – ksolaima@umbc.edu

ML TOOLKITS

Toolkit Basics

- Machine learning involves working with data
 - analyzing, manipulating, transforming, ...
- More often than not, it's numeric or has a natural numeric representation
- Natural language text is an exception, but this too can have a numeric representation
- A common data model is as a N-dimensional matrix or tensor
- These are supported in Python via libraries

Typical Python Libraries

numpy, scipy

- Basic mathematical libraries for dealing with matrices and scientific/mathematical functions

pandas, matplotlib

- Libraries for data science & plotting

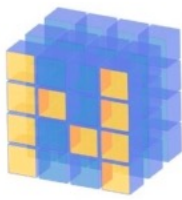
sklearn (scikit-learn)

- A whole bunch of implemented classifiers

torch (pytorch) and tensorflow

- Frameworks for building neural networks

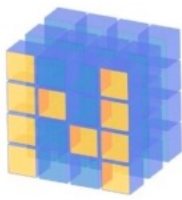
Lots of
documentation
available for all
of these online!



What is Numpy?

- NumPy supports features needed for ML
 - Typed N-dimensional arrays (matrices/tensors)
 - Fast numerical computations (matrix math)
 - High-level math functions
- Python does numerical computations slowly and lacks an efficient matrix representation
- 1000 x 1000 matrix multiply
 - **Python triple loop takes > 10 minutes!**
 - **Numpy takes ~0.03 seconds**

NumPy Arrays Can Represent ..



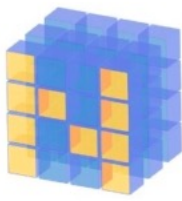
Structured lists of numbers

- **Vectors**
- **Matrices**
- Images
- Tensors
- Convolutional Neural Networks

$$\begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}$$

NumPy Arrays Can Represent ..

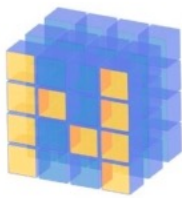


Structured lists of numbers

- Vectors
- Matrices
- **Images**
- Tensors
- Convolutional Neural Networks

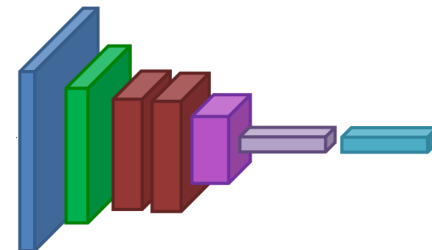
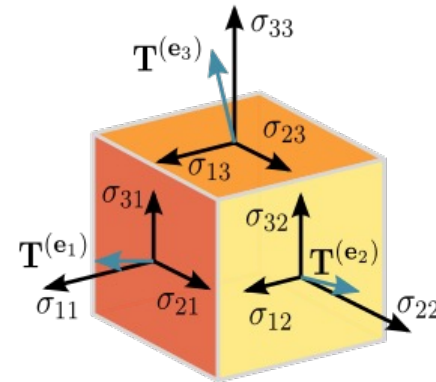


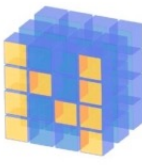
NumPy Arrays Can Represent ..



Structured lists of numbers

- Vectors
- Matrices
- Images
- **Tensors**
- **Convolutional Neural Networks**





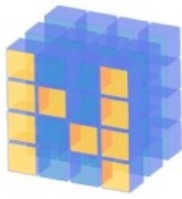
NumPy Arrays, Basic Properties

```
>>> import numpy as np
>>> a= np.array([[1,2,3],[4,5,6]],dtype=np.float32)
>>> print(a.ndim, a.shape, a.dtype)
2 (2, 3) float32
>> print(a)
[[1. 2. 3.]
 [4. 5. 6.] ]
```

Arrays:

1. Can have any number of dimensions, including zero (a scalar)
2. Are **typed**: np.uint8, np.int64, np.float32, np.float64
3. Are **dense**: each element of array exists and has the same type

NumPy Array Indexing, Slicing



`a[0,0]` # top-left element

`a[0,-1]` # first row, last column

`a[0,:]` # first row, all columns

`a[:,0]` # first column, all rows

`a[0:2,0:2]` # 1st 2 rows, 1st 2 columns

Notes:

- Zero-indexing
- Multi-dimensional indices are comma-separated)
- Python notation for slicing



SciPy

- SciPy builds on the NumPy array object
- Adds additional mathematical functions and *sparse arrays*
- **Sparse array:** one where most elements = 0
- An efficient representation only implicitly encodes the non-zero values
- Access to a missing element returns 0



SciPy sparse array use case

- NumPy and SciPy arrays are numeric
- We can represent a document's content by a vector of features
- Each feature is a possible word
- A feature's value might be any of:
 - TF: number of times it occurs in the document;
 - TF-IDF: ... normalized by how common the word is
 - and maybe normalized by document length ...

SciPy sparse array use case



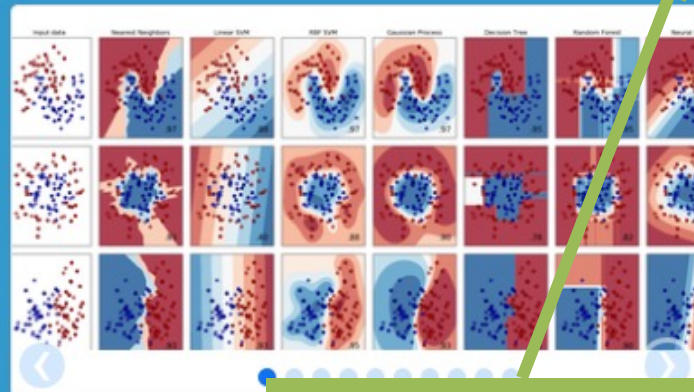
- Maybe only model 50k most frequent words found in a document collection, ignoring others
- Assign each unique word an index (e.g., dog:137)
 - Build python dict **w** from vocabulary, so `w['dog']=137`
- The sentence “the dog chased the cat”
 - Would be a *numPy* vector of length 50,000
 - Or a *sciPy* sparse vector of length 4
- An 800-word news article may only have 100 unique words; [The Hobbit](#) has about 8,000

SciPy Tutorial

- [Introduction](#)
- [Basic functions](#)
- [Special functions \(scipy.special\)](#)
- [Integration \(scipy.integrate\)](#)
- [Optimization \(scipy.optimize\)](#)
- [Interpolation \(scipy.interpolate\)](#)
- [Fourier Transforms \(scipy.fft\)](#)
- [Signal Processing \(scipy.signal\)](#)
- [Linear Algebra \(scipy.linalg\)](#)
- [Sparse eigenvalue problems with ARPACK](#)
- [Compressed Sparse Graph Routines \(scipy.sparse.csgraph\)](#)
- [Spatial data structures and algorithms \(scipy.spatial\)](#)
- [Statistics \(scipy.stats\)](#)
- [Multidimensional image processing \(scipy.ndimage\)](#)
- [File IO \(scipy.io\)](#)

More on SciPy

See the [SciPy tutorial](#) Web pages



Documentation online

scikit-learn

Machine Learning in Python

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Many tutorials

Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.

Algorithms: SVM, nearest neighbors, random forest, ... — Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, ridge regression, Lasso, ... — Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, ... — Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization. — Examples

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning

Modules: grid search, cross validation, metrics. — Examples

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.

Modules: preprocessing, feature extraction. — Examples

How easy is this?

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

```
>>> from sklearn.datasets import load_iris
```

```
>>> from sklearn.linear_model import LogisticRegression
```

```
>>> X, y = load_iris(return_X_y=True)
```

features on
data

labels

```
>>> clf = LogisticRegression(random_state=0).fit(X, y)
```

DATA & EVALUATION

Central Question: How Well Are We Doing?

Classification

- Precision, Recall, F1
- Accuracy
- Log-loss
- ROC-AUC
- ...

Regression

- (Root) Mean Square Error
- Mean Absolute Error
- ...

Clustering

- Mutual Information
- V-score
- ...

*the **task**: what kind of problem are you solving?*

Evaluation Metrics

Classification

- Precision, Recall, F1
- Accuracy
- Log-loss
- ROC-AUC
- ...

Regression

- (Root) Mean Square Error
- Mean Absolute Error
- ...

Clustering

- Mutual Information
- V-score
- ...

This does not have to be the same thing as the loss function you optimize

the **task**: what kind of problem are you solving?

Evaluation methodology (1)

Standard methodology:

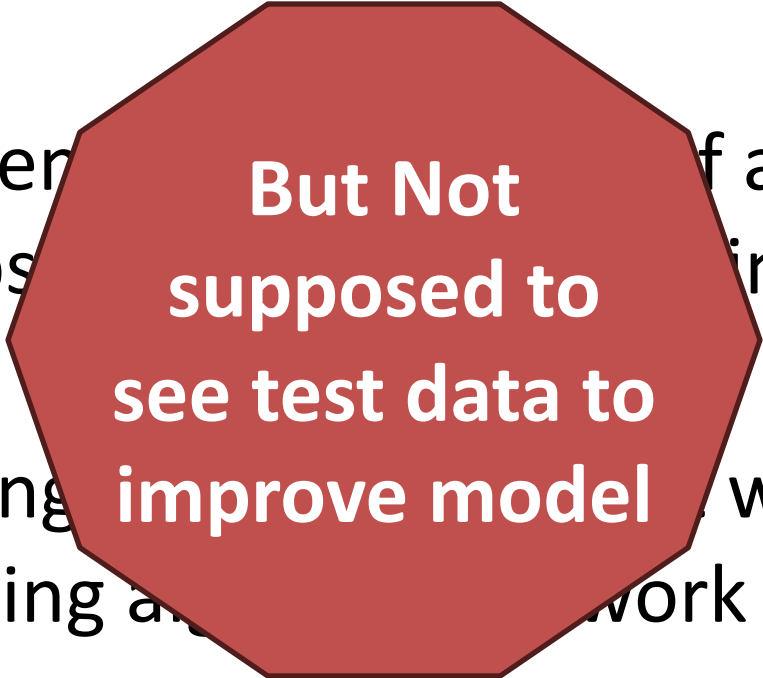
1. Collect large set of examples with correct classifications (aka ground truth data)
2. Randomly divide collection into two disjoint sets: **training** and **test** (*e.g., via a 90-10% split*)
3. Apply learning algorithm to **training** set giving hypothesis H
4. Measure performance of H on the held-out **test** set

Evaluation methodology (2)

- **Important: keep the training and test sets disjoint!**
- Study efficiency & robustness of algorithm: repeat steps 2-4 for different training sets & training set sizes
- On modifying algorithm, restart with step 1 to avoid evolving algorithm to work well on just this collection

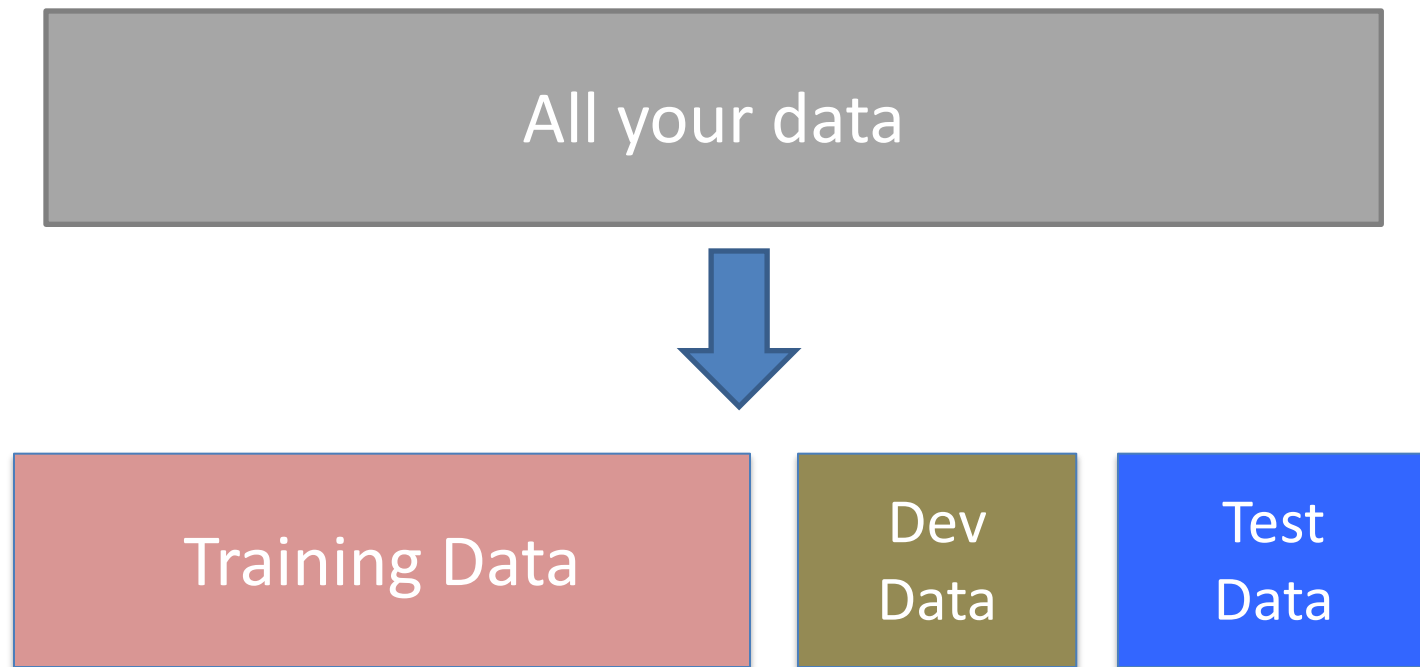
Evaluation methodology (2)

- **Important: keep the training and test sets disjoint!**
- Study efficiency of algorithm:
repeat steps 1-3 with different training sets & training set size
- On modifying algorithm with step 1 to avoid evolving algorithm to work well on just this collection

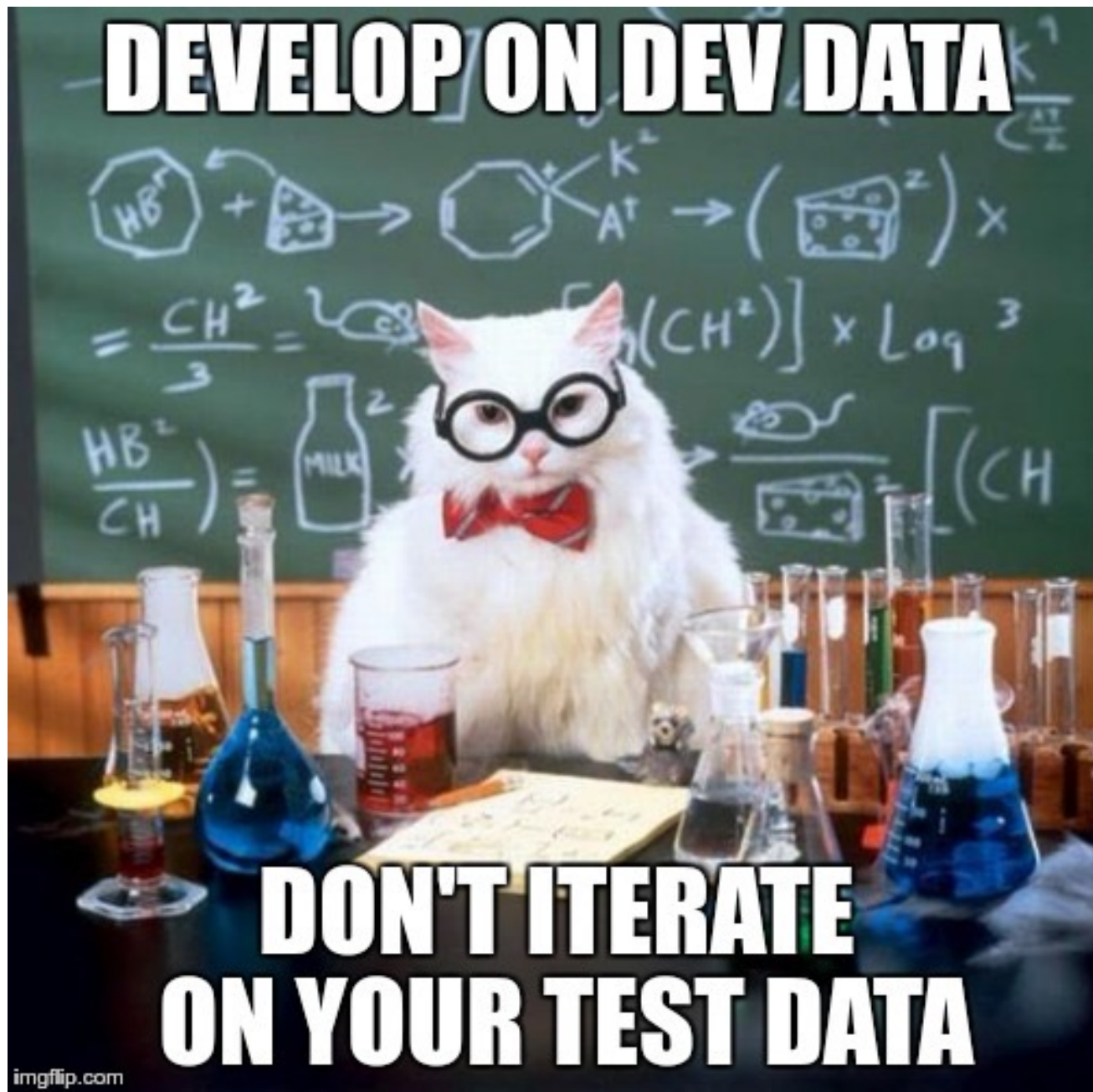


**But Not
supposed to
see test data to
improve model**

Experimenting with Machine Learning Models



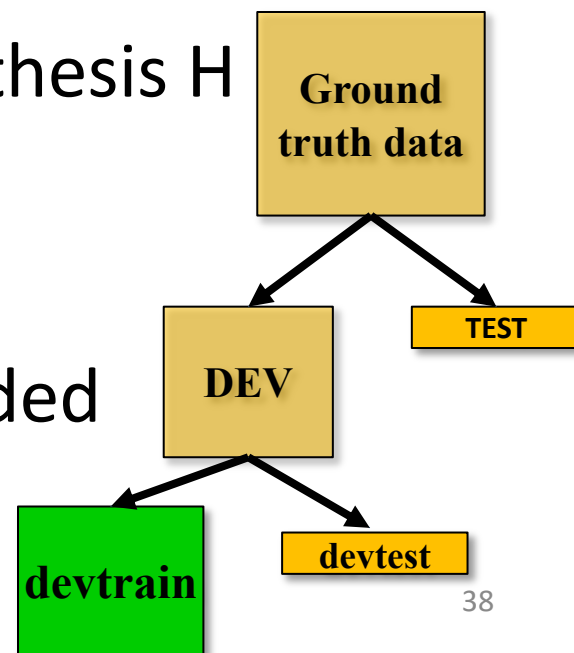
Rule #1



Evaluation methodology (3)

Common variation on methodology:

1. Collect set of examples with correct classifications
2. Randomly divide it into two disjoint sets:
development & ***test***; further divide development into ***devtrain*** & ***devtest***
3. Apply ML to *devtrain*, creating hypothesis H
4. Measure performance of H w.r.t.
devtest data
5. Modify approach, repeat 3-4 as needed
6. Final test on *test* data

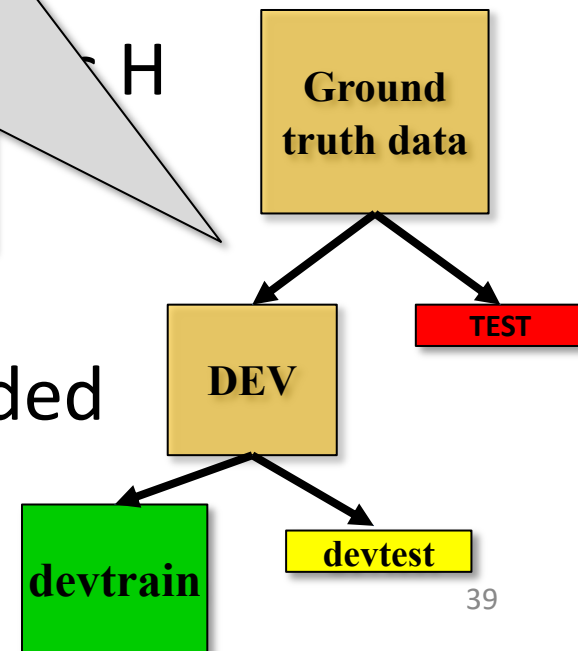


Evaluation methodology (4)

- Consider the following classification sets:
1. **devtest** data used for evaluation during system **development**
 2. When all development has ended, **test** data used for **final evaluation**
 3. Ensures final system not influenced by test data
 4. If more development needed, get new dataset!

devtest data

5. Modify approach, repeat 3-4 as needed
6. Final test on *test* data

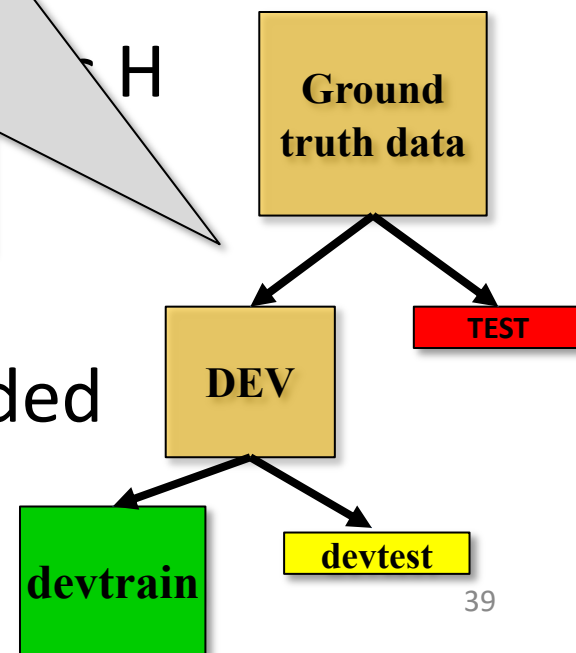


Evaluation methodology (4)

- Consider the following classification sets:
1. **devtest** data used for evaluation during system **development**
 2. When all development has ended, **test** data used for **final evaluation**
 3. Ensures final system not influenced by test data
 4. If more development needed, get new dataset!

devtest data

5. Modify approach, repeat 3-4 as needed
6. Final test on *test* data



Evaluation with different dev-test

```
def train_and_test(clf, data, start, end):  
    # Splitting the data  
    train_data = data.subset(examples=range(start, end))  
    test_data = data.subset(examples=range(end))  
  
    # Prepare training and testing sets  
    X_train = train_data.inputs  
    y_train = train_data.values  
    X_test = test_data.inputs  
    y_test = test_data.values  
  
    # Initialize Decision Tree classifier  
    clf = clf.fit(X_train, y_train)  
  
    # Predict on test set  
    y_pred = clf.predict(X_test)  
  
    # Calculate accuracy  
    accuracy = accuracy_score(y_test, y_pred)  
  
    return accuracy
```

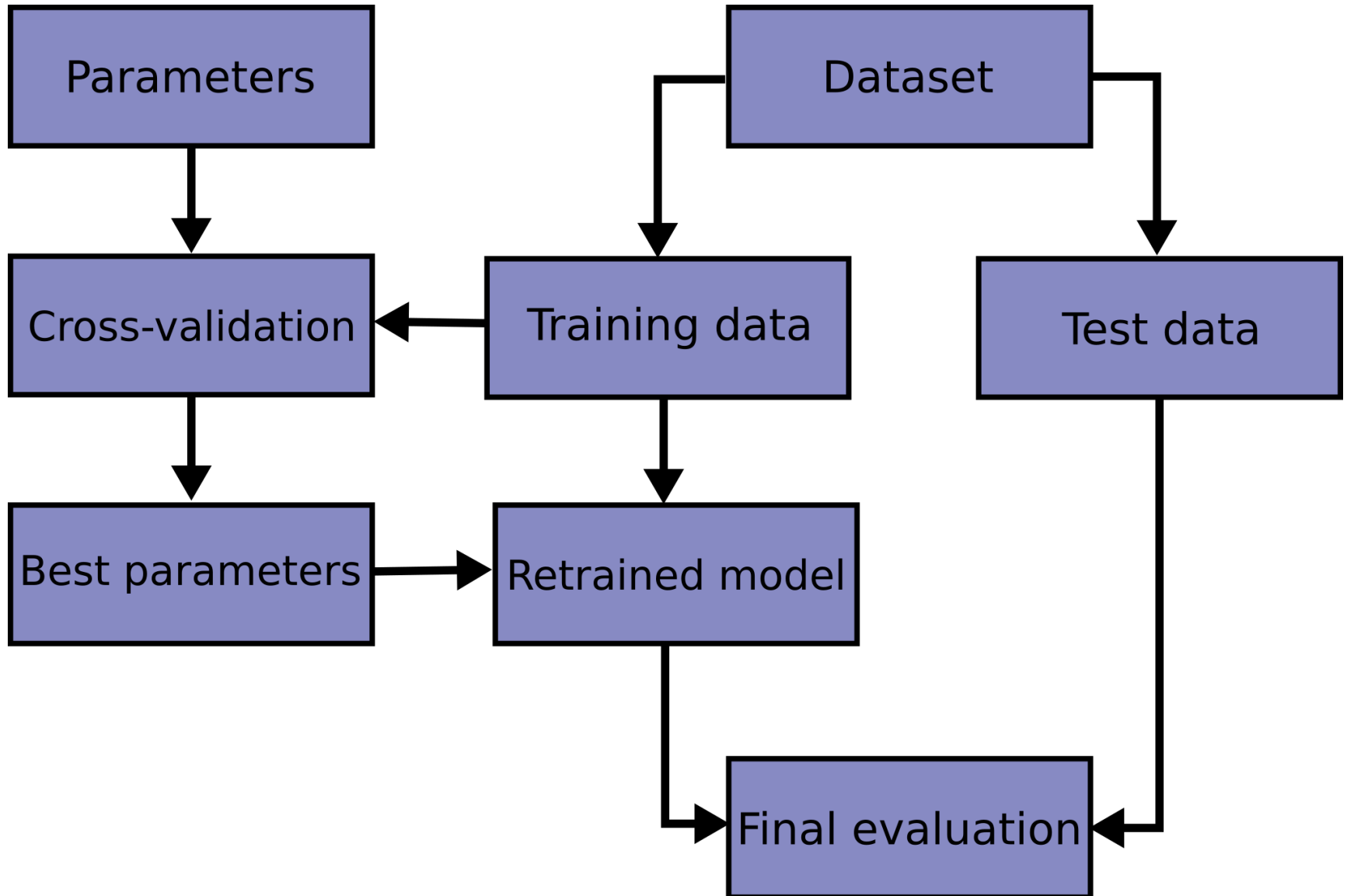
```
# Initialize Decision Tree classifier  
clf = DecisionTreeClassifier()  
  
# Perform train and test for different ranges  
accuracy_1 = train_and_test(clf, zoo, 0, 10)  
# >>> 1.0  
  
accuracy_2 = train_and_test(clf, zoo, 90, 100)  
# >>> 0.80000000000000004  
  
accuracy_3 = train_and_test(clf, zoo, 90, 101)  
# >>> 0.81818181818181823  
  
accuracy_4 = train_and_test(clf, zoo, 80, 90)  
# >>> 0.90000000000000002
```

Evaluation with different dev-test

- We hold out 10 data items for test; train on the other 91; show the accuracy on the test data
- Doing this **four times for different test subsets** shows accuracy from 80% to 100%
- **What's the true accuracy of our approach?**

K-fold Cross Validation

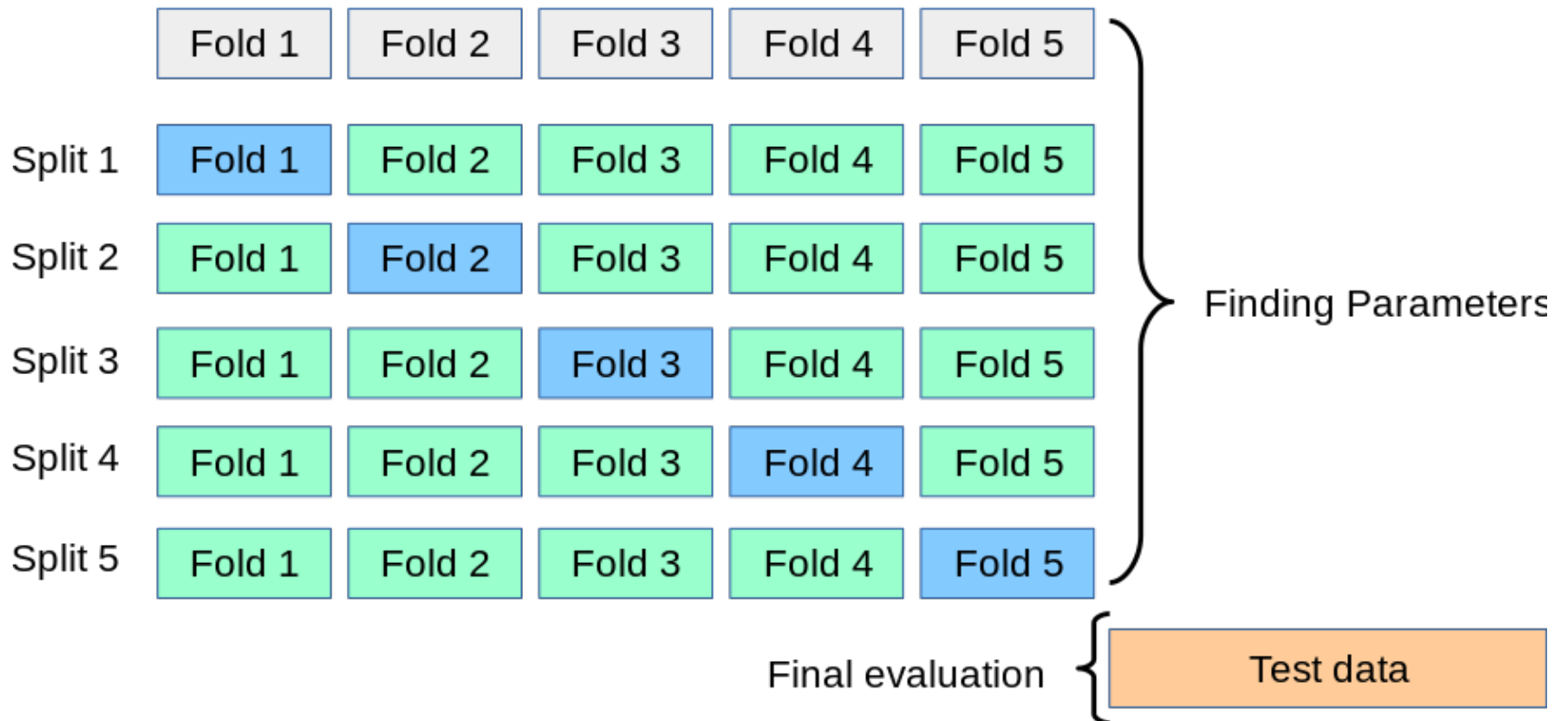
- **Problems:**
 - getting *ground truth* data expensive
 - need different test data for each test
 - experiments needed to find right *feature space* & parameters for ML algorithms
- **Goal:** minimize training+test data needed
- **Idea:** split training data into K subsets; use K-1 for *training* and one for *development testing*
- Repeat K times and average performance
- Common K values are 5 and 10



All Data

Training data

Test data



sklearn.model_selection.KFold

```
class sklearn.model_selection.KFold(n_splits=5, *, shuffle=False, random_state=None)
```

[\[source\]](#)

K-Fold cross-validator.

Provides train/test indices to split data in train/test sets. Split dataset into k consecutive folds (without shuffling by default).

Each fold is then used once as a validation while the k - 1 remaining folds form the training set.

Read more in the [User Guide](#).

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to [Visualizing cross-validation behavior in scikit-learn](#)

Parameters:

n_splits : int, default=5

Number of folds. Must be at least 2.

Changed in version 0.22: n_splits default value changed from 3 to 5.

shuffle : bool, default=False

Whether to shuffle the data before splitting into batches. Note that the samples within each split will not be shuffled.

random_state : int, RandomState instance or None, default=None

When `shuffle` is True, `random_state` affects the ordering of the indices, which controls the randomness of each fold. Otherwise, this parameter has no effect. Pass an int for reproducible output across multiple function calls. See [Glossary](#).

sklearn.model_selection.StratifiedKFold

```
class sklearn.model_selection.StratifiedKFold(n_splits=5, *, shuffle=False, random_state=None)
```

[\[source\]](#)

Stratified K-Fold cross-validator.

Provides train/test indices to split data in train/test sets.

This cross-validation object is a variation of KFold that returns stratified folds. The folds are made by preserving the percentage of samples for each class.

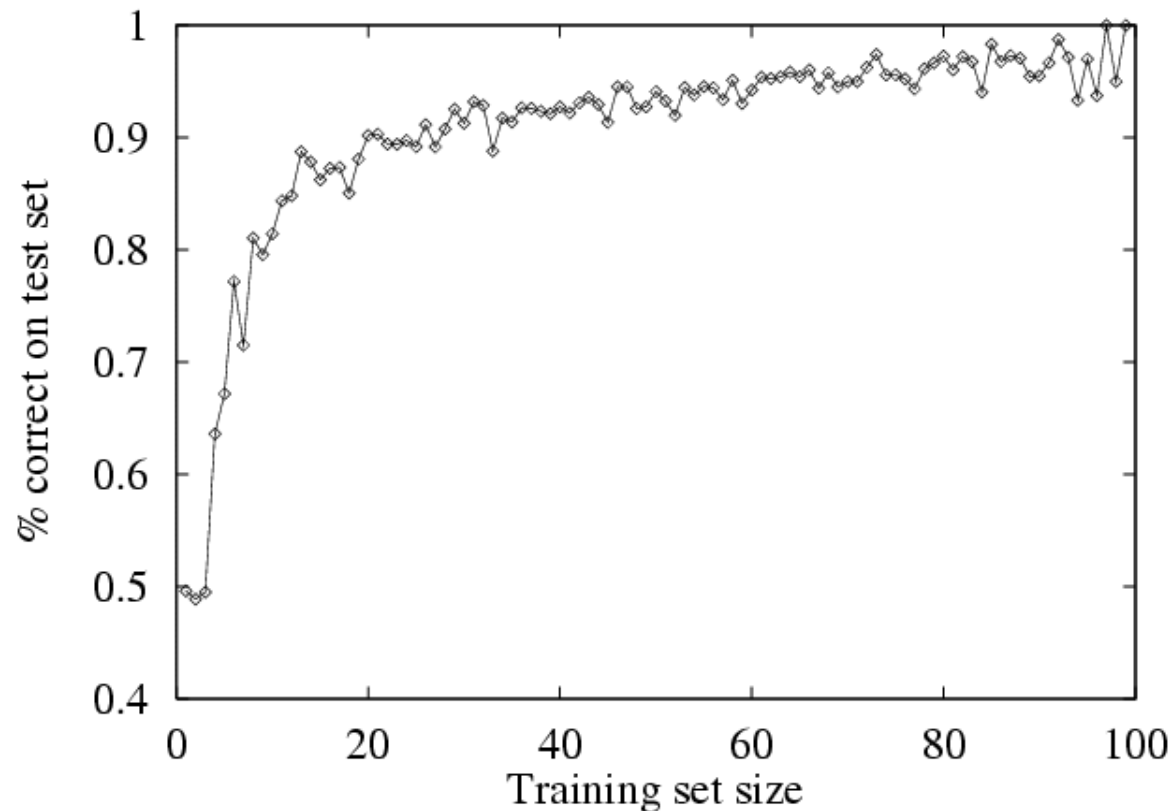
- Takes class information into account to avoid building folds with **imbalanced class distributions** (for binary or multiclass classification tasks).

Leave one out Cross Validation

- Sklearn also has a *LeaveOneOut* function that Provides train/test indices to split data in train/test sets. Each sample is used once as a test set (singleton) while the remaining samples form the training set.
- *LeaveOneOut()* is equivalent to *KFold(n_splits=n)* where n is the number of samples.
- K-fold cross validation can be too pessimistic, since it only trains with 80% or 90% of the data
- The leave one out evaluation is an alternative

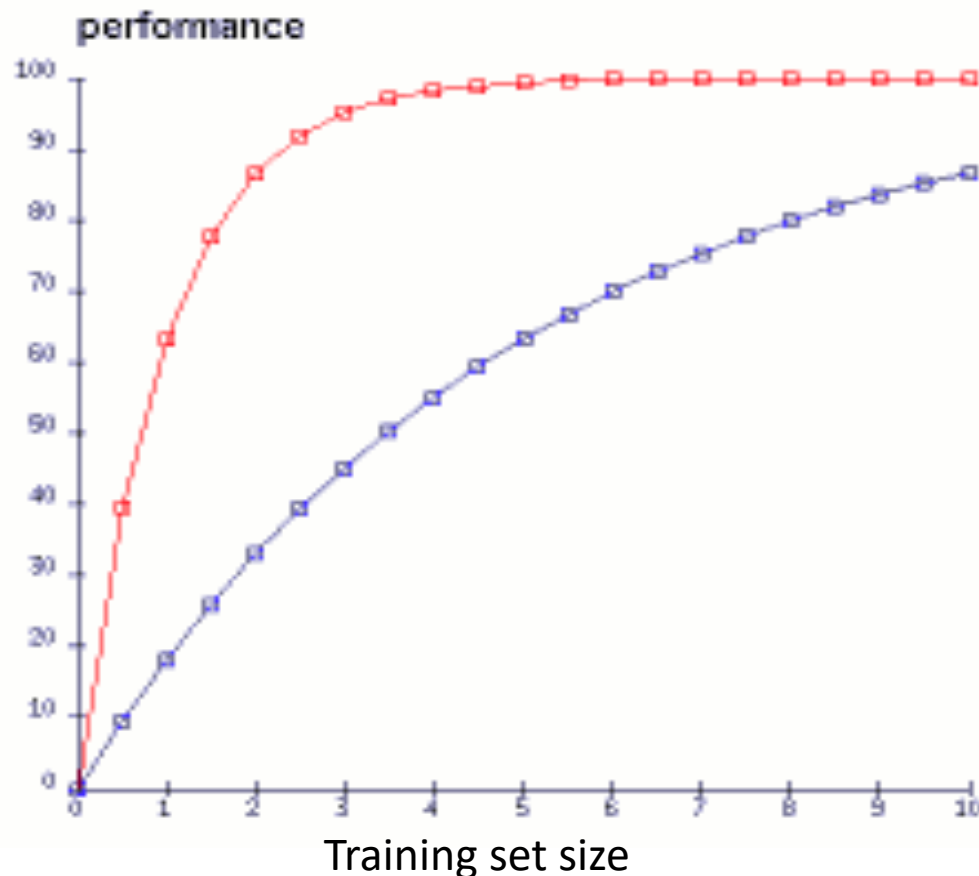
Learning curve (1)

A [learning curve](#) shows accuracy on test set as a function of training set size or (for neural networks) number of epochs



Learning curve

- When evaluating ML algorithms, steeper learning curves are better
- They represents faster learning with less data

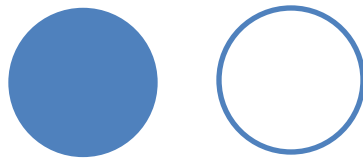


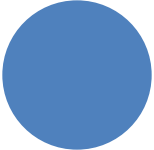
Here the system with the red curve is better since it requires less data to achieve desired accuracy

EVALUATION METRICS

Classification Evaluation: the 2-by-2 contingency table

Let's assume there are two classes/labels



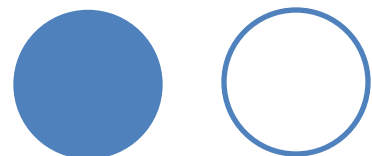
Assume  is the “positive” label

Given X , our classifier predicts either label

$$p(\text{solid blue circle} | X) \text{ vs. } p(\text{empty blue circle} | X)$$

Classification Evaluation: the 2-by-2 contingency table

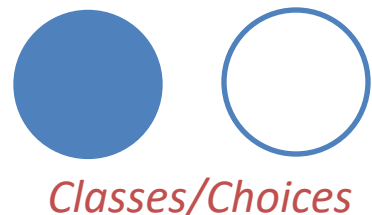
<i>What label does our system predict? (↓)</i>	<i>What is the actual label?</i>	
	Actually Correct	Actually Incorrect
Selected/ Guessed		
Not selected/ not guessed		



Classes/Choices

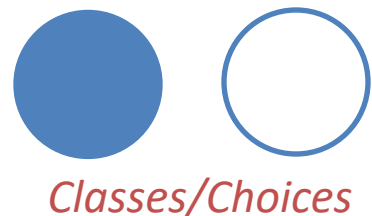
Classification Evaluation: the 2-by-2 contingency table

<i>What label does our system predict? (↓)</i>	<i>What is the actual label?</i>	
	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive  (TP)  <i>Actual</i> <i>Guessed</i>	
Not selected/ not guessed		



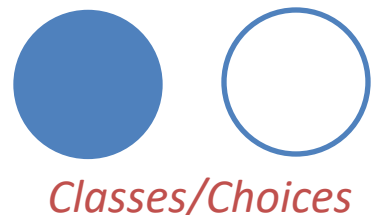
Classification Evaluation: the 2-by-2 contingency table

<i>What is the actual label?</i>		
<i>What label does our system predict? (↓)</i>	Actually Correct	Actually Incorrect
	True Positive  (TP)  <i>Actual</i> <i>Guessed</i>	False Positive  (FP)  <i>Actual</i> <i>Guessed</i>
Selected/ Guessed		
Not selected/ not guessed		



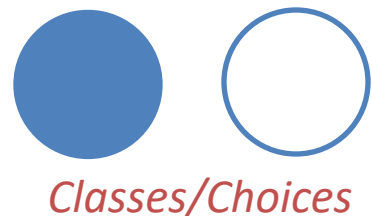
Classification Evaluation: the 2-by-2 contingency table

<i>What is the actual label?</i>		
<i>What label does our system predict? (↓)</i>	Actually Correct	Actually Incorrect
	True Positive  (TP)  <i>Actual</i> <i>Guessed</i>	False Positive  (FP)  <i>Actual</i> <i>Guessed</i>
Selected/ Guessed		
Not selected/ not guessed	False Negative  (FN)  <i>Actual</i> <i>Guessed</i>	



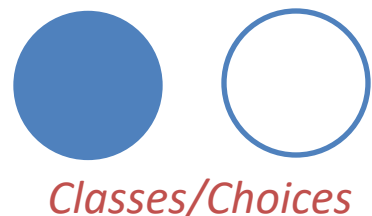
Classification Evaluation: the 2-by-2 contingency table

	<i>What is the actual label?</i>	
<i>What label does our system predict? (↓)</i>	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive  (TP)  <i>Actual</i> <i>Guessed</i>	False Positive  (FP)  <i>Actual</i> <i>Guessed</i>
Not selected/ not guessed	False Negative  (FN)  <i>Actual</i> <i>Guessed</i>	True Negative  (TN)  <i>Actual</i> <i>Guessed</i>



Classification Evaluation: the 2-by-2 contingency table

<i>What is the actual label?</i>			
<i>What label does our system predict? (↓)</i>	Actually Correct	Actually Incorrect	
	True Positive  (TP)  <i>Actual</i> <i>Guessed</i>	False Positive  (FP)  <i>Actual</i> <i>Guessed</i>	
Selected/ Guessed			
Not selected/ not guessed	False Negative  (FN)  <i>Actual</i> <i>Guessed</i>	True Negative  (TN)  <i>Actual</i> <i>Guessed</i>	



Construct this table by *counting*
the number of TPs, FPs, FNs, TNs

Contingency Table Example

Predicted:					
Actual:					

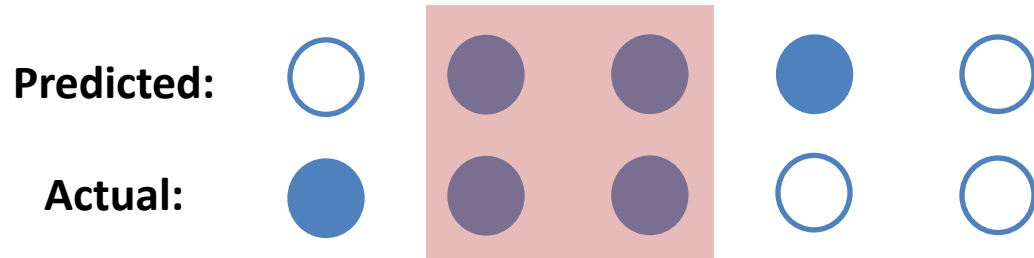
Contingency Table Example

Predicted:     

Actual:     

What is the actual label?		
What label does our system predict? (↓)	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP)	False Positive (FP)
Not selected/ not guessed	False Negative (FN)	True Negative (TN)

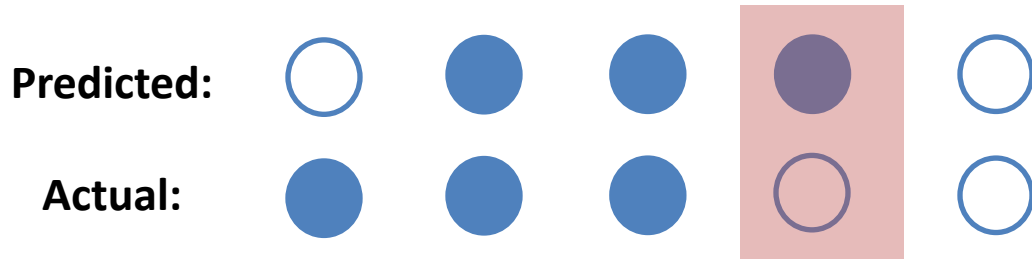
Contingency Table Example



	What is the actual label?	
What label does our system predict? (↓)	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP) = 2	False Positive (FP)
Not selected/ not guessed	False Negative (FN)	True Negative (TN)

61

Contingency Table Example



	What is the actual label?	
What label does our system predict? (↓)	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP) = 2	False Positive (FP) = 1
Not selected/ not guessed	False Negative (FN)	True Negative (TN)

Contingency Table Example

Predicted:

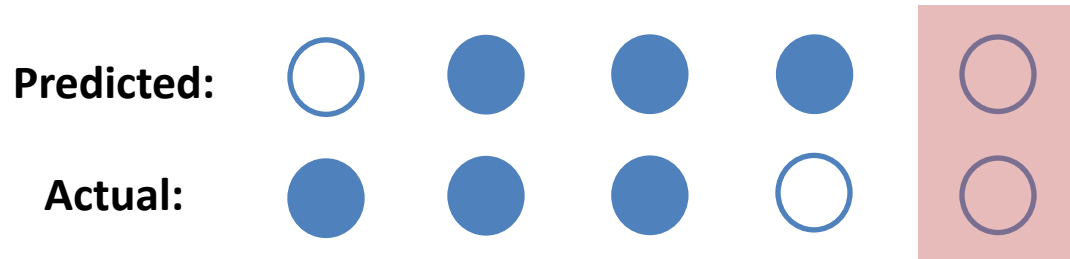


Actual:



	What is the actual label?	
What label does our system predict? (↓)	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP) = 2	False Positive (FP) = 1
Not selected/ not guessed	False Negative (FN) = 1	True Negative (TN)

Contingency Table Example



	<i>What is the actual label?</i>	
<i>What label does our system predict? (↓)</i>	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP) = 2	False Positive (FP) = 1
Not selected/ not guessed	False Negative (FN) = 1	True Negative (TN) = 1

Contingency Table Example

Predicted:     

Actual:     

What is the actual label?		
What label does our system predict? (↓)	Actually Correct	Actually Incorrect
Selected/ Guessed	True Positive (TP) = 2	False Positive (FP) = 1
Not selected/ not guessed	False Negative (FN) = 1	True Negative (TN) = 1

Classification Evaluation: Accuracy, Precision, and Recall

Accuracy: % of items correct

$$\frac{TP + TN}{TP + FP + FN + TN}$$

	Actually Correct	Actually Incorrect
Selected/Guessed	True Positive (TP)	False Positive (FP)
Not select/not guessed	False Negative (FN)	True Negative (TN)

Classification Evaluation:

Accuracy, Precision, and Recall

Accuracy: % of items correct

$$\frac{TP + TN}{TP + FP + FN + TN}$$

Precision: % of selected items that are correct

$$\frac{TP}{TP + FP}$$

	Actually Correct	Actually Incorrect
Selected/Guessed	True Positive (TP)	False Positive (FP)
Not select/not guessed	False Negative (FN)	True Negative (TN)

Classification Evaluation:

Accuracy, Precision, and Recall

Accuracy: % of items correct

$$\frac{TP + TN}{TP + FP + FN + TN}$$

Precision: % of selected items that are correct

$$\frac{TP}{TP + FP}$$

Recall: % of correct items that are selected

$$\frac{TP}{TP + FN}$$

	Actually Correct	Actually Incorrect
Selected/Guessed	True Positive (TP)	False Positive (FP)
Not select/not guessed	False Negative (FN)	True Negative (TN)

Classification Evaluation:

Accuracy, Precision, and Recall

Accuracy: % of items correct

$$\frac{TP + TN}{TP + FP + FN + TN}$$

Precision: % of selected items that are correct

$$\frac{TP}{TP + FP}$$

Min: 0 😞

Max: 1 😊

Recall: % of correct items that are selected

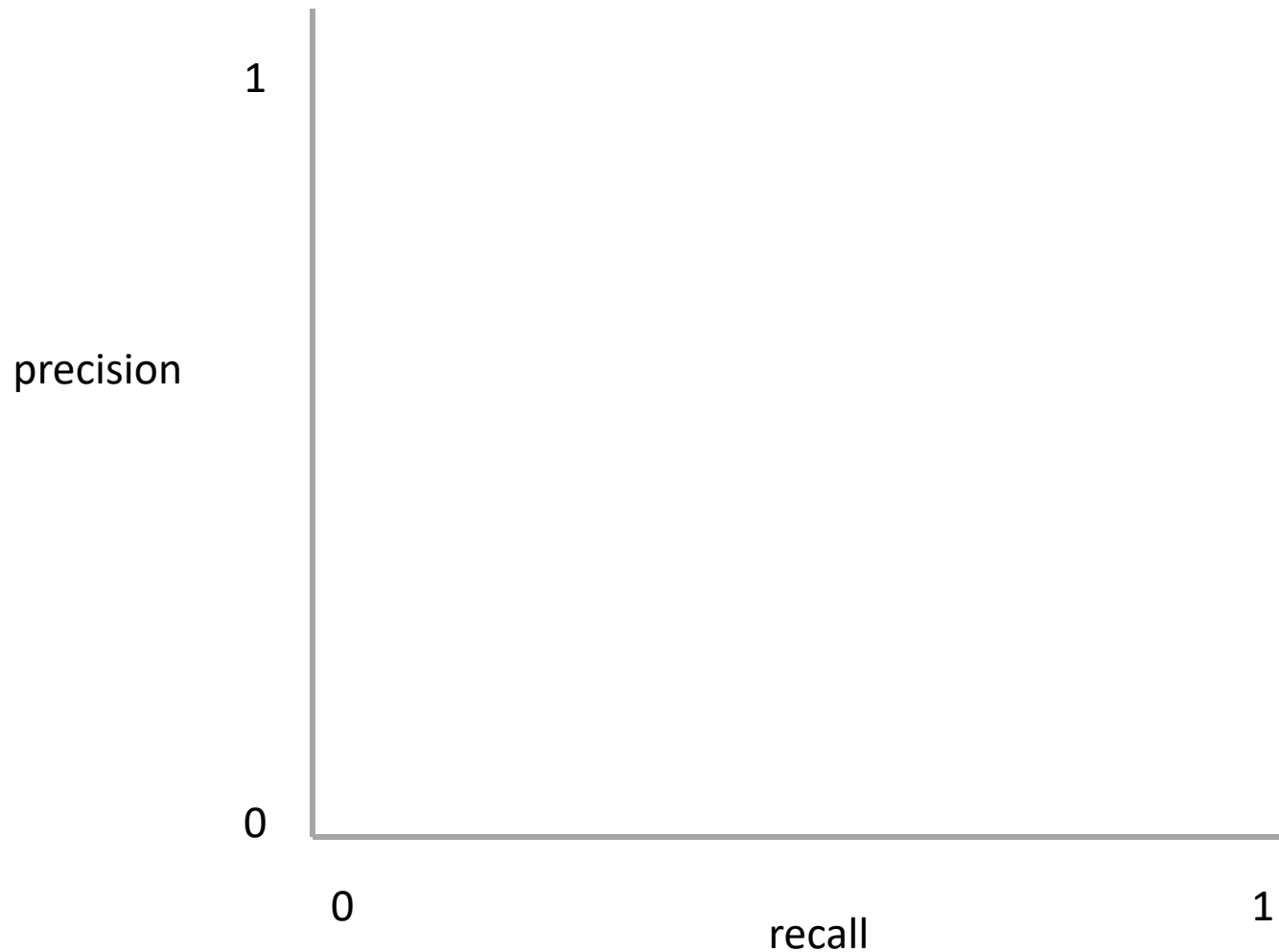
$$\frac{TP}{TP + FN}$$

	Actually Correct	Actually Incorrect
Selected/Guessed	True Positive (TP)	False Positive (FP)
Not select/not guessed	False Negative (FN)	True Negative (TN)

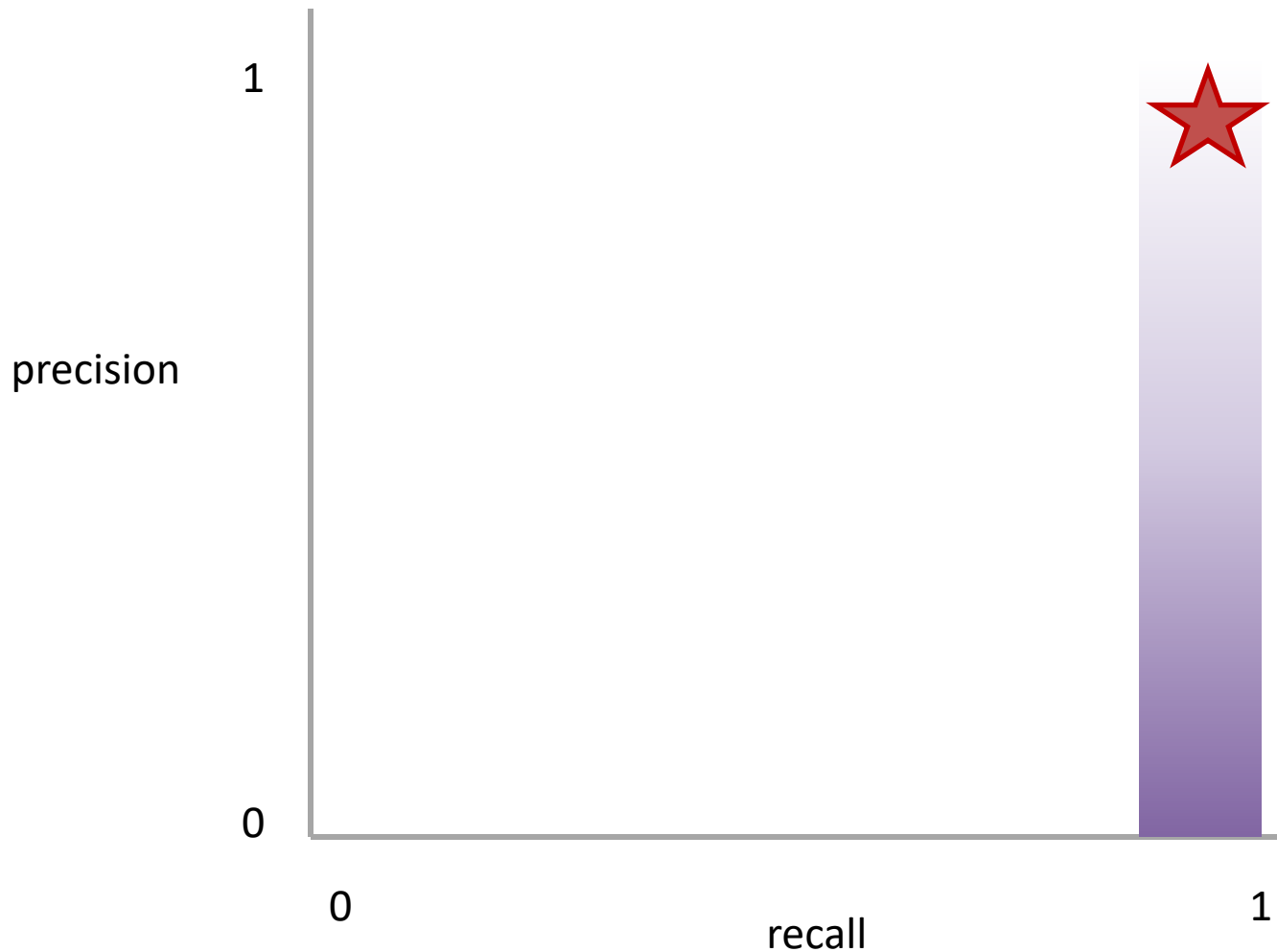
Precision and Recall Present a Tradeoff

Q: Where do you
want your ideal

model ?



Precision and Recall Present a Tradeoff



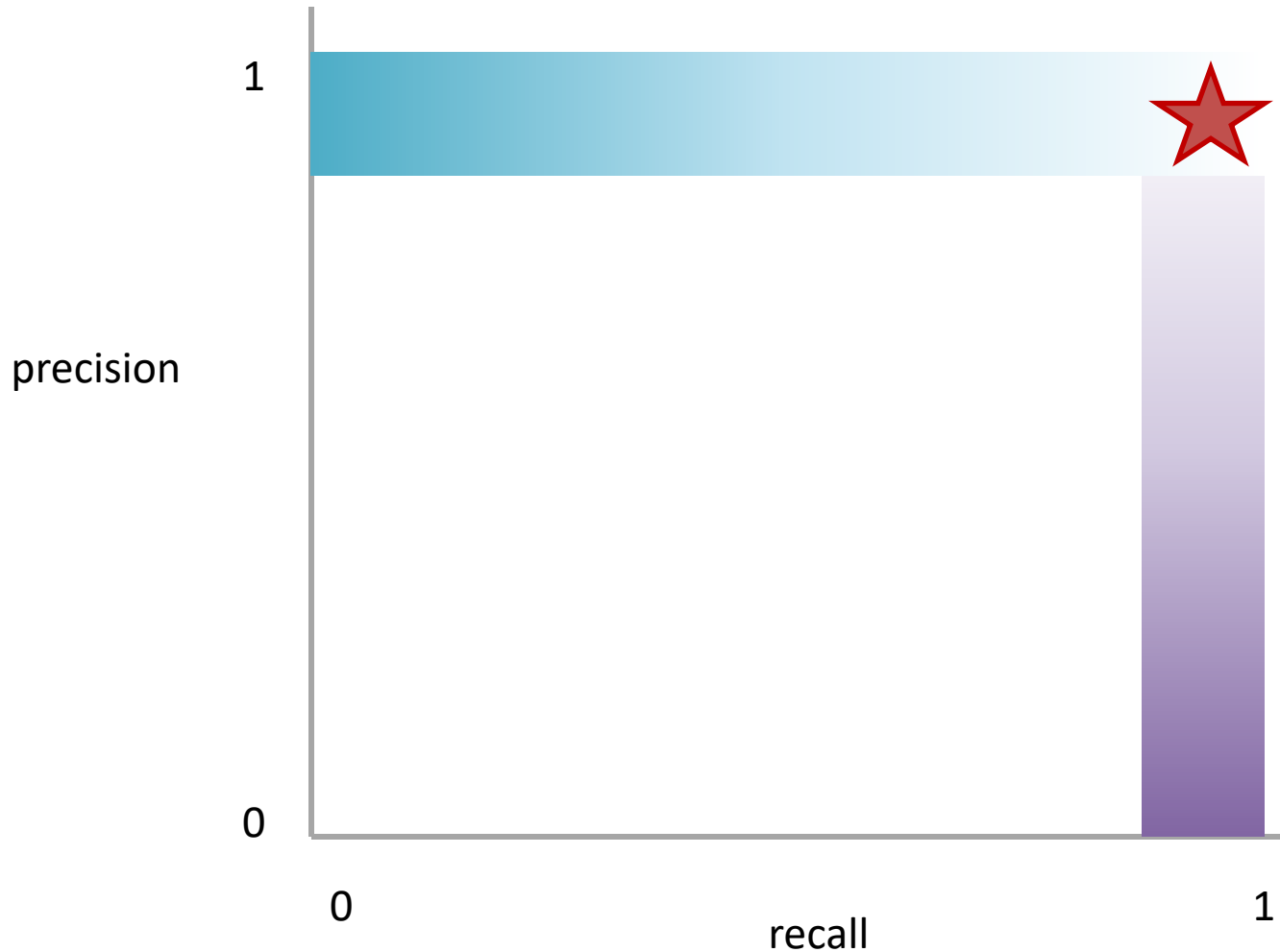
Q: Where do you want your ideal

model ?

Q: You have a model that always identifies correct instances. Where on this graph is it?

Q: You have a model that only make correct predictions. Where on this graph is it?

Precision and Recall Present a Tradeoff



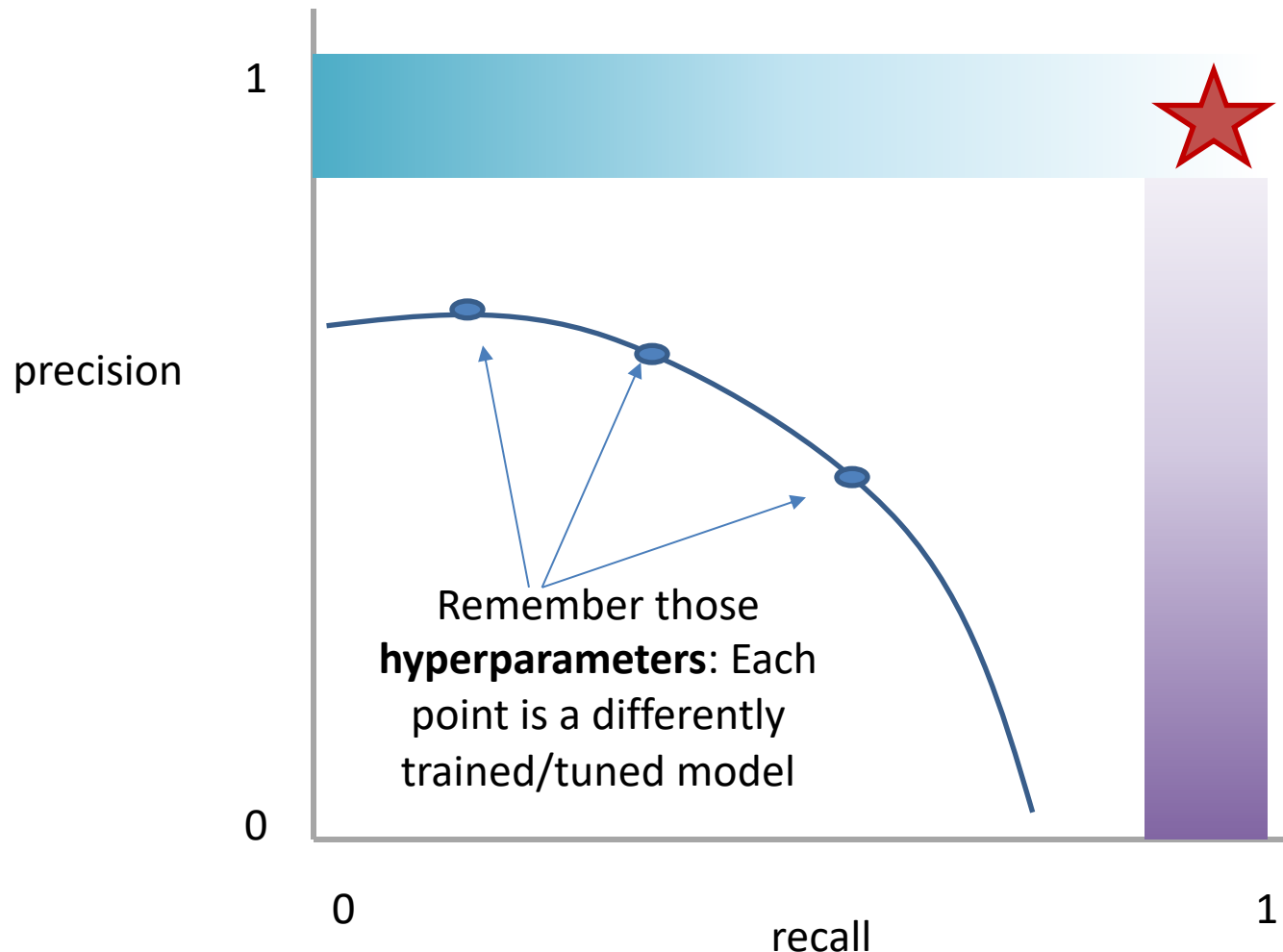
Q: Where do you want your ideal

model ?

Q: You have a model that always identifies correct instances. Where on this graph is it?

Q: You have a model that only make correct predictions. Where on this graph is it?

Precision and Recall Present a Tradeoff



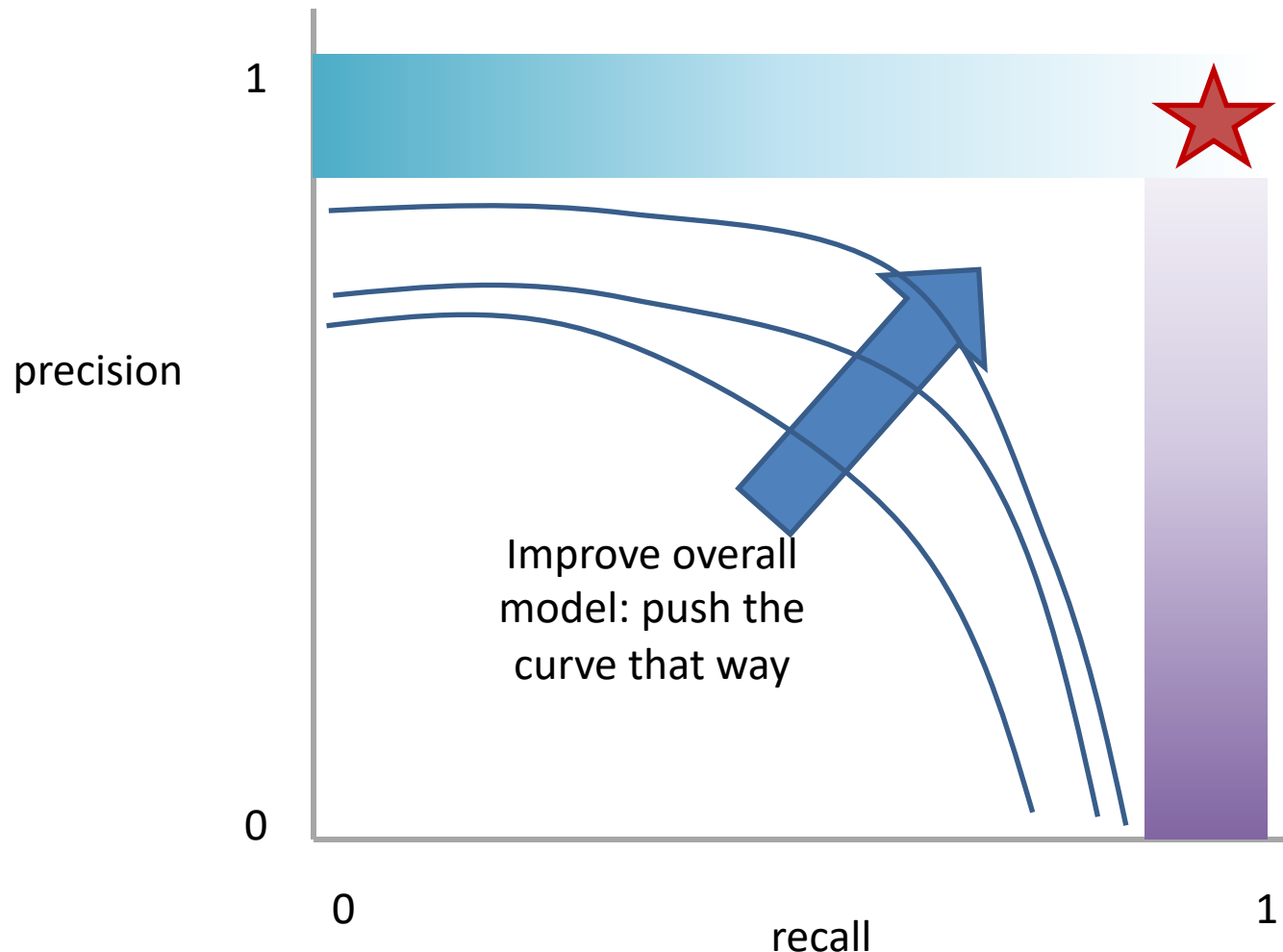
Q: Where do you want your ideal **model** ?

Q: You have a **model** that always identifies correct instances. Where on this graph is it?

Q: You have a **model** that only make correct predictions. Where on this graph is it?

Idea: measure the tradeoff between precision and recall

Precision and Recall Present a Tradeoff



Q: Where do you want your ideal **model** ?

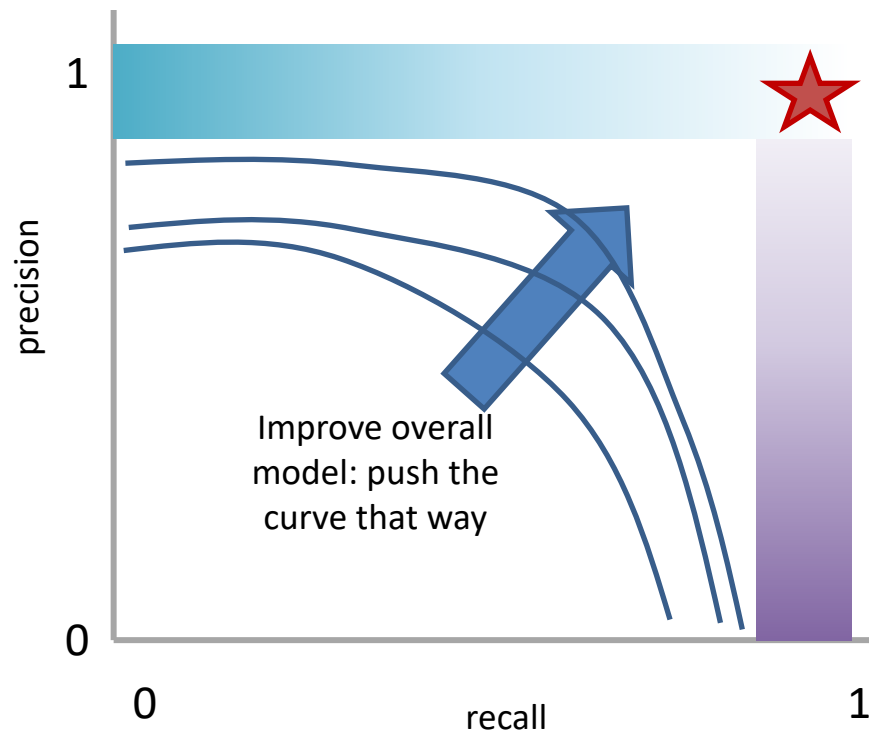
Q: You have a **model** that always identifies correct instances. Where on this graph is it?

Q: You have a **model** that only make correct predictions. Where on this graph is it?

Idea: measure the tradeoff between precision and recall

Measure this Tradeoff: Area Under the Curve (AUC)

AUC measures the area under this tradeoff curve

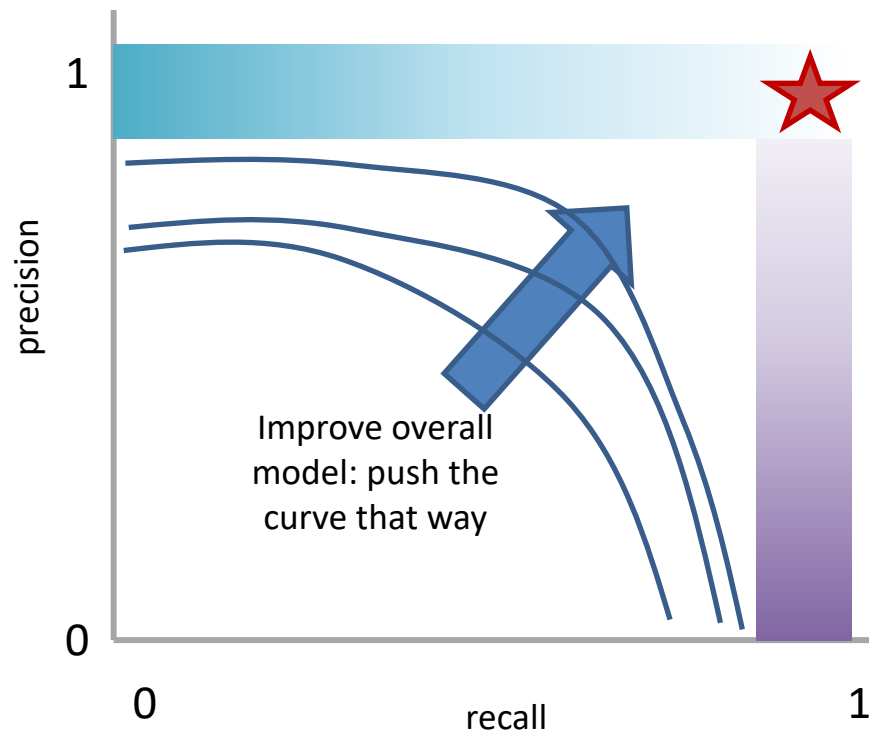


Min AUC: 0 😞

Max AUC: 1 😊

Measure this Tradeoff: Area Under the Curve (AUC)

AUC measures the area under this tradeoff curve



Min AUC: 0 😞

Max AUC: 1 😊

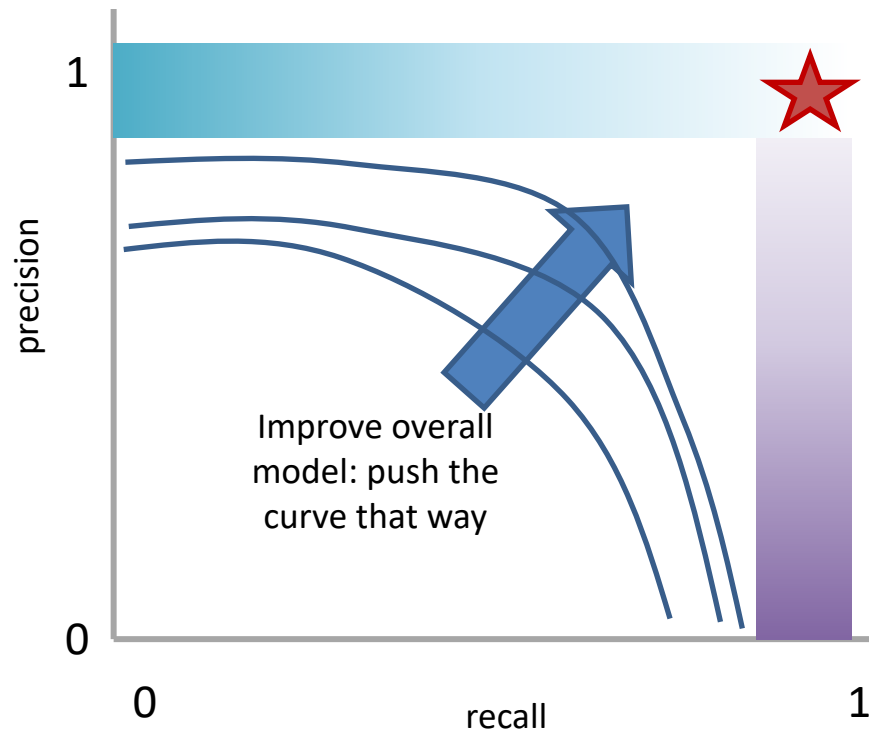
1. Computing the curve

You need true labels & predicted labels with some score/confidence estimate

Threshold the scores and for each threshold compute precision and recall

Measure this Tradeoff: Area Under the Curve (AUC)

AUC measures the area under this tradeoff curve



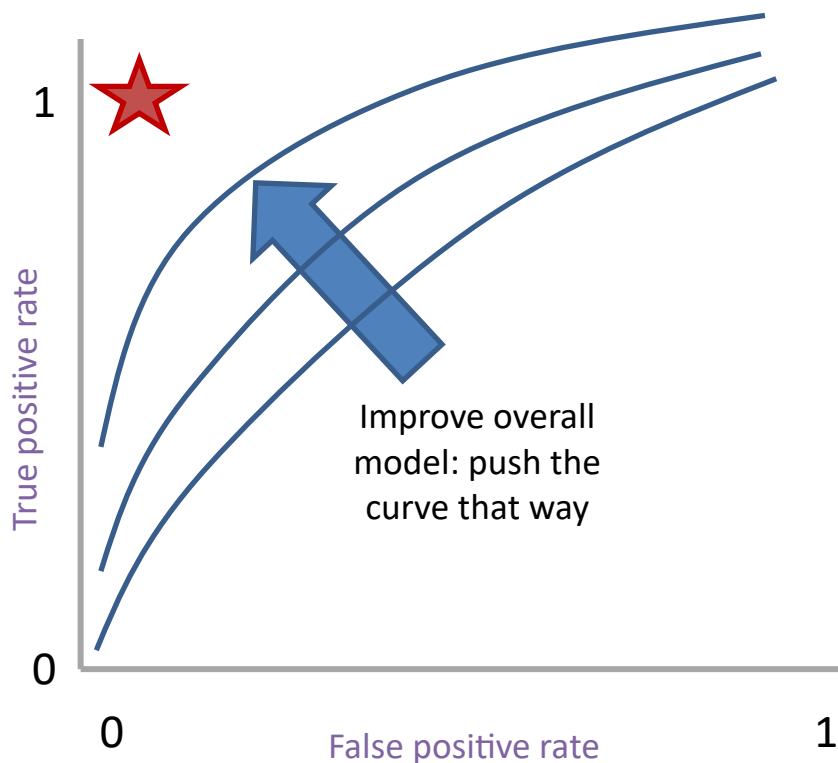
Min AUC: 0 😞

Max AUC: 1 😊

1. Computing the curve
You need true labels & predicted labels with some score/confidence estimate
Threshold the scores and for each threshold compute precision and recall
2. Finding the area
How to implement: trapezoidal rule (& others)

In practice: external library like the *sklearn.metrics* module

Measure A Slightly Different Tradeoff: ROC-AUC



Min ROC-AUC: 0.5 🙄

Max ROC-AUC: 1 😊

AUC measures the area under this tradeoff curve

1. Computing the curve
You need true labels & predicted labels with some score/confidence estimate
Threshold the scores and for each threshold compute metrics
2. Finding the area
How to implement: trapezoidal rule (& others)

In practice: external library like the `sklearn.metrics` module

Main variant: ROC-AUC

Same idea as before but with some
flipped metrics

A combined measure: F

Weighted (harmonic) average of **P**recision & **R**ecall

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}}$$

A combined measure: F

Weighted (harmonic) average of **P**recision & **R**ecall

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} = \frac{(1 + \beta^2) * P * R}{(\beta^2 * P) + R}$$


*algebra
(not important)*

A combined measure: F

Weighted (harmonic) average of **P**recision & **R**ecall

$$F = \frac{(1 + \beta^2) * P * R}{(\beta^2 * P) + R}$$

Balanced F1 measure: $\beta=1$

$$F_1 = \frac{2 * P * R}{P + R}$$

P/R/F in a Multi-class Setting: Micro- vs. Macro-Averaging

If we have more than one class, how do we combine multiple performance measures into one quantity?

Macroaveraging: Compute performance for each class, then average.

Microaveraging: Collect decisions for all classes, compute contingency table, evaluate.

P/R/F in a Multi-class Setting: Micro- vs. Macro-Averaging

Macroaveraging: Compute performance for each class, then average.

$$\text{macroprecision} = \sum_c \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} = \sum_c \text{precision}_c$$

Microaveraging: Collect decisions for all classes, compute contingency table, evaluate.

$$\text{microprecision} = \frac{\sum_c \text{TP}_c}{\sum_c \text{TP}_c + \sum_c \text{FP}_c}$$

P/R/F in a Multi-class Setting: Micro- vs. Macro-Averaging

Macroaveraging: Compute performance for each class, then average.

when to prefer the macroaverage?

$$\text{macroprecision} = \sum_c \frac{TP_c}{TP_c + FP_c} = \sum_c \text{precision}_c$$

Microaveraging: Collect decisions for all classes, compute contingency table, evaluate.

when to prefer the microaverage?

$$\text{microprecision} = \frac{\sum_c TP_c}{\sum_c TP_c + \sum_c FP_c}$$

Micro- vs. Macro-Averaging: Example

Class 1

	Truth : yes	Truth : no
Classifier: yes	10	10
Classifier: no	10	970

Class 2

	Truth : yes	Truth : no
Classifier: yes	90	10
Classifier: no	10	890

Micro Ave. Table

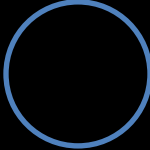
	Truth : yes	Truth : no
Classifier: yes	100 (90+10)	20 (10+10)
Classifier: no	20	1860

Macroaveraged precision: $(10/10+10) + (90/90+10)/2 = (0.5 + 0.9)/2 = 0.7$

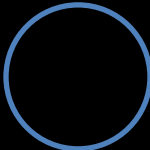
Microaveraged precision: $100/100+20 = .83$

Microaveraged score is dominated by score on frequent classes

Confusion Matrix: Generalizing the 2-by-2 contingency table

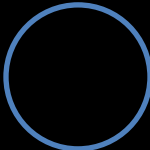
		Correct Value		
				
Guessed Value		#	#	#
		#	#	#
		#	#	#

Confusion Matrix: Generalizing the 2-by-2 contingency table

		Correct Value		
				
Guessed Value		80	9	11
		7	86	7
		2	8	9

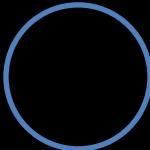
Q: Is this a good result?

Confusion Matrix: Generalizing the 2-by-2 contingency table

		Correct Value		
				
Guessed Value		30	40	30
		25	30	50
		30	35	35

Q: Is this a good result?

Confusion Matrix: Generalizing the 2-by-2 contingency table

		Correct Value		
				
Guessed Value		7	3	90
		4	8	88
		3	7	90

Q: Is this a good result?